

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет о выполнении лабораторной работы №5
по дисциплине «Математическая логика и теория формальных языков»
«Реализация детерминированного левого анализатора
грамматики LL(1) »
Вариант 19

Студент,
группы 5130201/30101

_____ Мартыненко А. П.

Преподаватель

_____ Востров А. В.

«_____» _____ 2026г.

Содержание

Введение	3
1 Математическое описание	4
1.1 Иерархия Хомского	4
1.2 LL(k)-грамматики и LL(1)-грамматики	4
1.3 LR(k)-грамматики	5
2 Построение множеств	7
3 Определение типа грамматики	9
3.1 Приведение грамматики к виду LL(1)	9
3.2 Проверка грамматики на принадлежность классу LR(k) . . .	10
3.2.1 Таблица состояний LALR(1)-анализатора	11
3.2.2 LR(0)	12
3.2.3 SLR	12
3.2.4 LALR(1)	13
3.2.5 LR(1)	13
4 Упрощение грамматики	15
4.1 Итоговые правила	15
4.2 Таблица выбора	15
4.3 Синтаксическая диаграмма	16
4.4 Заданные семантические действия	17
5 Особенности реализации	18
5.1 Структуры данных	18
5.2 Инициализация	18
5.3 Генератор цепочек	19
5.4 Анализатор цепочек	20
6 Результаты работы программы	22
Заключение	24
Список использованной литературы	25

Введение

В лабораторной работе необходимо:

1. Построить множества FIRST и FOLLOW для каждого нетерминала грамматики и таблицу выбора (lookup table).
2. Построить матрицу отношений предшествования.
3. Реализовать детерминированный левый или правый анализатор (проверку принадлежности цепочки грамматике) и генератор цепочек.
4. Назначить семантические действия части заданных продукций.

Заданная грамматика для варианта 19:

$S \rightarrow SbBc$

$B \rightarrow cD$

$S \rightarrow BdD$

$B \rightarrow \epsilon$

$D \rightarrow DBa$

$D \rightarrow \epsilon$

1 Математическое описание

1.1 Иерархия Хомского

Порождающая грамматика Хомского: $G = (T, N, S, R)$, где

- T - конечное множество терминалов (слова языка).
- N - конечное множество нетерминалов $T \cap N = \emptyset$.
- $S \in N$ - начальный терминал.
- R - множество правил вывода вида $\alpha \rightarrow \beta$.

Согласно иерархии Хомского, формальные грамматики делятся на 4 типа в соответствии с накладываемыми на их правила ограничениями:

1. Тип 0 (неограниченные грамматики): продукции имеют вид $\alpha \rightarrow \beta$, где α содержит хотя бы один нетерминал.
2. Тип 1 (контекстно-зависимые грамматики): продукции имеют вид $\alpha A \beta \rightarrow \alpha \gamma \beta$, где $A \in N$, $\alpha, \beta \in (T \cup N)^*$, $\gamma \in (T \cup N)^+$.
3. Тип 2 (контекстно-свободные грамматики): все продукции имеют вид $A \rightarrow \beta$, где $A \in N$, $\beta \in (T \cup N)^+$ (при этом левые части всех продукций содержат только один нетерминальный символ.).
4. Тип 3 (автоматные грамматики): продукции имеют вид $A \rightarrow aB$, $A \rightarrow Ba$ или $A \rightarrow a$, где $A, B \in N$, $a \in T$.

Соотношение между типами грамматик: $\mathcal{L}_3 \subset \mathcal{L}_2 \subset \mathcal{L}_1 \subset \mathcal{L}_0$, где $\mathcal{L}_i$ — класс языков, порождаемых грамматиками типа i .

Для цепочек КС-языков вывод их из изначального символа удобно представляется деревом. Для любой порождающей КС-грамматики существуют полиномиальные алгоритмы (алгоритмы синтаксического анализа), позволяющие для произвольной цепочки над входным словарем восстановить дерево вывода этой цепочки.

1.2 LL(k)-грамматики и LL(1)-грамматики

LL(k)-грамматика — это класс контекстно-свободных грамматик. LL(k)-грамматики позволяют выполнять нисходящий синтаксический анализ, просматривая входную цепочку слева направо для построения канонического левого вывода. Здесь k указывает, сколько символов из непрочитанной части входной цепочки используется для принятия решений. LL(1)-грамматики, например, используют только один символ впереди.

Грамматика G является $LL(k)$ -грамматикой, если для любых двух левых выводов:

$$S \Rightarrow^* wAx \Rightarrow w\alpha x \Rightarrow^* wu$$

$$S \Rightarrow^* wAx \Rightarrow w\beta x \Rightarrow^* wv$$

где wAx - текущая сентенциальная форма, $wu(wv)$ - распознаваемая цепочка (w - распознанная часть цепочки, $u(v)$ - нераспознанная), $A \in N$, A заменяется по правилам $A \rightarrow \alpha$, $A \rightarrow \beta$ где $u, v \in T^*$, $\alpha, \beta \in (T \cup N)^*$, из условия $FIRST_k(u) = FIRST_k(v)$ следует, что $\alpha = \beta$.

Для построения $LL(k)$ грамматик можно определить функции:

Функции $FIRST(\alpha)$ и $FOLLOW(A)$ помогают определить:

- $FIRST(\alpha)$ - с каких символов могут начинаться цепочки, выводимые из α . Если множества $FIRST(\alpha)$ и $FIRST(\beta)$ не пересекаются, можно однозначно выбрать между правилами $A \rightarrow \alpha$ и $A \rightarrow \beta$.
- $FOLLOW(A)$ - какие символы могут следовать за A в любых формах предложения. Если $FIRST(\alpha)$ и $FOLLOW(A)$ не пересекаются, можно выбрать между $A \rightarrow \alpha$ и $A \rightarrow \epsilon$.

Грамматика является $LL(k)$ тогда и только тогда, когда для каждого нетерминала A и любых двух различных продукций $A \rightarrow \alpha$ и $A \rightarrow \beta$ выполняется

$$FIRST_k(\alpha FOLLOW_k(A)) \cap FIRST_k(\beta FOLLOW_k(A)) = \emptyset$$

Грамматика является $LL(1)$, если для каждого нетерминала A с продукциями $A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$ выполняются условия:

1. $FIRST(\alpha_i) \cap FIRST(\alpha_j) = \emptyset$ для всех $i \neq j$;
2. если $\alpha_i \Rightarrow^* \epsilon$, то $FIRST(\alpha_j) \cap FOLLOW(A) = \emptyset$ для всех $j \neq i$,

то есть множества выбора для всех альтернатив каждого нетерминала не пересекаются.

1.3 LR(k)-грамматики

$LR(k)$ -грамматика — это наиболее широкий класс контекстно-свободных грамматик.

$LR(k)$ -грамматики позволяют выполнять восходящий синтаксический анализ, просматривая входную цепочку слева направо для построения правого канонического вывода. Здесь k указывает, сколько символов из непрочитанной части входной цепочки используется для принятия решений. На практике рассматривается $k=1$ или $k=0$.

LR(0) называется такая грамматика, для которой парсер знает о свертке, основываясь только на текущем состоянии автомата.

SLR(1) - это усовершенствованная LR(0), которая смотрит на один символ вперед и принимает решение о свертке на основе множества FOLLOW.

LALR - тип грамматики, который тоже использует один символ просмотра вперед, но вычисляет его для каждой позиции в правиле.

LR(1) - самый мощный тип грамматики. Он смотрит на один символ вперед, при этом каждое состояние автомата хранит точное множество допустимых символов для конкретной позиции.

Множества грамматик, языки которых могут быть распознаны анализаторами, строго включаются одно в другое: LR(0)-грамматики $\subset$ SLR(1)-грамматики $\subset$ LALR(1)-грамматики $\subset$ LR(1)-грамматики.

2 Построение множеств

Заданная грамматика:

$$S \rightarrow SbBc \mid BdD$$

$$B \rightarrow cD \mid \epsilon$$

$$D \rightarrow DBa \mid \epsilon$$

Построим множества FIRST и FOLLOW для изначальной грамматики.

Таблица 1. Множества FIRST и FOLLOW

Нетерминал	FIRST	FOLLOW
S	c, d	b, \$
B	c, ϵ	a, c, d
D	c, a, ϵ	a, b, c, d, \$

Таблица выбора представлена в таблице 2.

Таблица 2. Таблица выбора

	a	b	c	d	\$
S	ошибка	ошибка	$S \rightarrow SbBc$ $S \rightarrow BdD$	$S \rightarrow SbBc$ $S \rightarrow BdD$	ошибка
B	$B \rightarrow \epsilon$	ошибка	$B \rightarrow cD$ $B \rightarrow \epsilon$	$B \rightarrow \epsilon$	ошибка
D	$D \rightarrow DBa$ $D \rightarrow \epsilon$	$D \rightarrow \epsilon$	$D \rightarrow DBa$ $D \rightarrow \epsilon$	$D \rightarrow \epsilon$	$D \rightarrow \epsilon$

Матрица отношений предшествования:

Таблица 3. Матрица отношений предшествования

	a	b	c	d
a	>	>	=,>	>
b			<,<=	
c	<,>	<,>	<,>	>
d	<		<	

Для вывода нужны множества FIRSTVT и FOLLOWVT. FIRSTVT(A) - множество терминалов, которые могут быть первыми в строке, выводимой

из A . FOLLOWVT - множество терминалов, которые могут быть последними в строке, выводимой из A .

Таблица 4. Множества FIRSTVT и FOLLOWVT

Нетерминал	FIRSTVT	FOLLOWVT
S	c, d	a, c
B	c	a, c
D	a, c	a

Правила построения:

- Если в правиле $A \rightarrow \alpha ab\beta$, то $a = b$ (они стоят внутри одной конструкции).
- Если в правиле $A \rightarrow \alpha aB\beta$, то $a < x$, где $x \in FIRSTVT(B)$ (a уступает первому терминалу B).
- Если в правиле $A \rightarrow \alpha Bb\beta$, то $x > b$, где $x \in FOLLOWVT(B)$ (x имеет большее предшествование).

1. $S \rightarrow SbBc$

- Sb: $a > b, c > b$
- bB: $b < c$
- Bc: $a > c, c > c$
- bc: $b = c$

2. $S \rightarrow BdD$

- Bd: $a > d, c > d$
- dD: $d < a, d < c$

3. $B \rightarrow cD$: $c < a, c < c$

4. $D \rightarrow DBa$

- DB: $a = c$
- Ba: $a > a, c > a$

3 Определение типа грамматики

Заданная грамматика:

$$S \rightarrow SbBc \mid BdD$$

$$B \rightarrow cD \mid \epsilon$$

$$D \rightarrow DBa \mid \epsilon$$

3.1 Приведение грамматики к виду LL(1)

Сначала надо устранить левую рекурсию в двух правилах. Для этого введем новые нетерминалы S' и D' :

1. Правило $S \rightarrow SbBc \mid BdD$:

$$S \rightarrow BdDS'$$

$$S' \rightarrow bBcS' \mid \epsilon$$

2. Правило $D \rightarrow DBa \mid \epsilon$

$$D \rightarrow D'$$

$$D' \rightarrow BaD' \mid \epsilon$$

Так как $D \rightarrow D'$ - единичная продукция, можно преобразовать в $D \rightarrow BaD \mid \epsilon$

После устранения левой рекурсии правила приобрели вид:

$$S \rightarrow BdDS'$$

$$S' \rightarrow bBcS' \mid \epsilon$$

$$B \rightarrow cD \mid \epsilon$$

$$D \rightarrow BaD \mid \epsilon$$

В правилах $S' \rightarrow bBcS' \mid \epsilon$ и $D \rightarrow BaD \mid \epsilon$ возникает неоднозначность: В может раскрываться в cD или в ϵ . Для исправления:

1. В случае правила $S' \rightarrow bBcS' \mid \epsilon$ введем новый нетерминал B' :

$$S' \rightarrow bcB' \mid \epsilon$$

$$B' \rightarrow DcS' \mid S'$$

2. В случае правила $D \rightarrow BaD \mid \epsilon$ подставим В в D:

$$D \rightarrow cDaD \mid aD \mid \epsilon$$

В результате получились правила

$$S \rightarrow BdDS'$$

$$S' \rightarrow bcB'|\epsilon$$

$$B' \rightarrow DcS'|S'$$

$$B \rightarrow cD|\epsilon$$

$$D \rightarrow cDaD|aD|\epsilon$$

Таблица 5. Множества FIRST и FOLLOW

Нетерминал	FIRST	FOLLOW
S	c, d	\$
S'	b, ϵ	\$
B	c, ϵ	d
B'	c, a, b, ϵ	\$
D	c, a, ϵ	a, b, c, d, \$

Если построить множества выбора для D, то

$$FIRST = c, a, \epsilon$$

$$FOLLOW = a, b, c, d, \$$$

$$\text{То есть } FIRST(D) \cap FOLLOW(D) = c, a \neq \emptyset$$

- c может начинаться с вывода из D ($D \rightarrow cDaD$), и следовать за D в $B' \rightarrow DcS'$.
- a может начинаться с вывода из D ($D \rightarrow aD$) и следовать за D в $D \rightarrow cDaD$.

С такими правилами грамматика нельзя привести к виду LL(1) с помощью эквивалентных преобразований.

3.2 Проверка грамматики на принадлежность классу LR(k)

Построим матрицу отношений предшествования по правилам, полученным в попытке привести грамматику к виду LL(1):

Таблица 6. Матрица отношений предшествования

	a	b	c	d
a	<, >		<, >, =	
b			=	
c	<	<	<	
d	<		<	>

Грамматика не является грамматикой предшествования, так как в матрице отношений возникают конфликты.

3.2.1 Таблица состояний LALR(1)-анализатора

Таблица 7. Таблица состояний LALR(1)-анализатора

Состояние	LR-элементы	Действия
0	$S' \rightarrow .S$ $S \rightarrow .SbBc$ $S \rightarrow .BdD$ $B \rightarrow .cD$ $B \rightarrow .$	$c \rightarrow shift\ 3$ $d \rightarrow reduce\ B \rightarrow \epsilon$ $S \rightarrow 1$ $B \rightarrow 2$
1	$S' \rightarrow S.$ $S \rightarrow S.bBc$	$b \rightarrow shift\ 4$
2	$S \rightarrow B.dD$	$d \rightarrow shift\ 5$
3	$B \rightarrow c.D$ $D \rightarrow .DBa$ $D \rightarrow .$	$c, a, d \rightarrow reduce\ D \rightarrow \epsilon$ $D \rightarrow 6$
4	$S \rightarrow Sb.Bc$ $B \rightarrow .cD$ $B \rightarrow .$	$c \rightarrow shift\ 3$ $c \rightarrow reduce\ B \rightarrow \epsilon$ Конфликт shift/reduce
5	$S \rightarrow Bd.D$ $D \rightarrow .DBa$ $D \rightarrow .$	$c, a, b, \$ \rightarrow reduce\ D \rightarrow \epsilon$ $D \rightarrow 8$

6	$B \rightarrow cD.$ $D \rightarrow D.Ba$ $B \rightarrow .cD$ $B \rightarrow .$	$c \rightarrow shift\ 3$ $d, a \rightarrow reduce\ B \rightarrow cD$ $a \rightarrow reduce\ B \rightarrow \epsilon$ Конфликты s/r и r/r
7	$S \rightarrow SbB.c$	$c \rightarrow shift\ 10$
8	$S \rightarrow BdD.$ $D \rightarrow D.Ba$ $B \rightarrow .cD$ $B \rightarrow .$	$b, \$ \rightarrow reduce\ S \rightarrow BdD$ $c \rightarrow shift\ 3$ $a \rightarrow reduce\ B \rightarrow \epsilon$
9	$D \rightarrow DB.a$	$a \rightarrow shift\ 11$
10	$S \rightarrow SbBc.$	$b, \$ \rightarrow reduce\ S \rightarrow SbBc$
11	$D \rightarrow DBa.$	$a, b, c, d, \$ \rightarrow$ $reduce\ D \rightarrow DBa$

3.2.2 LR(0)

Грамматика не принадлежит классу LR(0), так как содержит конфликты, которые не могут разрешиться на основе текущего состояния автомата:

- В состоянии 4 и 6 при входном символе с одновременно можно и применить и сдвиг $B \rightarrow .cD$, и свертку $B \rightarrow .$
- В состоянии 6 ($B \rightarrow cD.$ и $D \rightarrow D.Ba$) при входном символе а применима и свертка $B \rightarrow cD.$, и свертка $B \rightarrow .$

Из-за конфликтов в состояниях 4 и 6 грамматика не является LR(0).

3.2.3 SLR

Для SLR(1) анализатора используется множество FOLLOW для ограничения свертки (reduce). Множества FOLLOW для нетерминалов B ($FOLLOW(B) = \{a, c, d\}$) и D ($FOLLOW(D) = \{a, b, c, d, \$\}$) пересекаются, из-за чего анализатор не может корректно определить следующее действие.

- В состоянии 4 ($S \rightarrow Sb.Bc$) при символе с одновременно можно и применить и сдвиг $B \rightarrow .cD$, и свертку $B \rightarrow .$, так как $c \in FOLLOW(B)$.

- В состоянии 6 анализатора при входном символе a анализатор одновременно допускает и свертку $B \rightarrow cD.$, и свертку $B \rightarrow \epsilon$, так как $a \in FOLLOW(B)$.

Из-за конфликтов в состояниях 4 и 6 грамматика не является SLR.

3.2.4 LALR(1)

В LALR(1) используются lookahead-множества, вычисленные для каждого LR(1)-элемента, после чего состояния с одинаковыми ядрами объединяются, а их lookahead-множества сливаются (то есть не различается, каким путём они пришли в это состояние).

- Для состояния 4: из контекста $S \rightarrow SbBc$ после B обязательно стоит c , поэтому lookahead для $B \rightarrow . = \{c\}$ и совпадает с lookahead для $B \rightarrow .cD = \{c\}$. При входном символе c одновременно можно применить и сдвиг $B \rightarrow .cD$, и свёртку $B \rightarrow \epsilon$.
- Для состояния 6: состояние достигается тремя путями разбора, которые LALR(1) объединяет в одно:
 - из контекста $S \rightarrow BdD$: после B стоит d , lookahead для $B \rightarrow cD$ равен $\{d\}$;
 - из контекста $S \rightarrow SbBc$: после B стоит c , lookahead для $B \rightarrow cD$ равен $\{c\}$;
 - из контекста $D \rightarrow DBa$: после B стоит a , lookahead для $B \rightarrow cD$ равен $\{a\}$.

После слияния lookahead для $B \rightarrow cD$ становится $\{a, c, d\}$. Lookahead для $B \rightarrow .$ в контексте $D \rightarrow D.Ba = \{a\}$. При входном символе a одновременно возможны свёртка $B \rightarrow cD$ и свёртка $B \rightarrow \epsilon$.

Из-за конфликтов в состояниях 4 и 6 грамматика не является LALR(1).

3.2.5 LR(1)

В LR(1) используются lookahead-множества, вычисленные для каждого элемента в каждом конкретном состоянии. Состояния с одинаковыми ядрами и разными lookahead остаются отдельными.

- Для состояния 4: из контекста $S \rightarrow SbBc$ lookahead для $B \rightarrow . = \{c\}$, lookahead для $B \rightarrow .cD = \{c\}$. При входном символе c одновременно возможны сдвиг $B \rightarrow .cD$ и свёртка $B \rightarrow \epsilon$.

- Для состояния 6: рассматриваем только контекст $D \rightarrow DBa$: lookahead для $B \rightarrow cD. = \{a\}$; lookahead для $B \rightarrow . = \{a\}$ (после B стоит a). При входном символе a возможны две свёртки, конфликт reduce/reduce сохраняется.

Из-за конфликтов в состояниях 4 и 6 грамматика не является LR(1).

4 Упрощение грамматики

Для сохранения душевного равновесия упростим грамматику до $D \rightarrow aD | \epsilon$ (так как существует правило $B \rightarrow \epsilon$, которое обратит в ϵ правило $D \rightarrow cDaD$). Тогда она будет соответствовать типу LL(1).

Реализация анализатора LL(1) более эффективна, чем анализатора LR, так как LR-анализатор требует построения таблиц action, goto, реализацию сложного механизма свертки. Также LR-анализатор хранит таблицы переходов по состояниям, что занимает больше памяти, чем реализация LL-анализатора. В LL-анализаторе в момент ошибки известен точный нетерминал, в то время как LR-анализатор обнаруживает ошибку по номеру состояния в стеке, и ее нужно дополнительно расшифровывать.

Так как реализация детерминированного анализатора в изначальной грамматике невозможна, разумнее упростить ее до вида LL(1).

4.1 Итоговые правила

Преобразованная грамматика вида LL(1):

$S \rightarrow BdDS'$

$S' \rightarrow bcB' | \epsilon$

$B \rightarrow cD | \epsilon$

$B' \rightarrow DcS' | S'$

$D \rightarrow aD | \epsilon$

Таблицы для FIRST и FOLLOW представлены в таблице 8.

Таблица 8. Множества FIRST и FOLLOW

Нетерминал	FIRST	FOLLOW
S	c, d	\$
S'	b, ϵ	\$
B	c, ϵ	d
B'	c, a, b, ϵ	\$
D	a, ϵ	d, b, c, \$

4.2 Таблица выбора

На рисунке 1 представлен пример вывода дерева цепочки cadabcas.

Таблица 9. Таблица выбора

	a	b	c	d	\$
S	ошибка	ошибка	$S \rightarrow BdDS'$	$S \rightarrow BdDS'$	ошибка
S'	ошибка	$S' \rightarrow bcB'$	ошибка	ошибка	$S' \rightarrow \epsilon$
B	ошибка	ошибка	$B \rightarrow cD$	$B \rightarrow \epsilon$	ошибка
B'	$B' \rightarrow DcS'$	$B' \rightarrow S'$	$B' \rightarrow DcS'$	ошибка	$B' \rightarrow S'$
D	$D \rightarrow aD$	$D \rightarrow \epsilon$	$D \rightarrow \epsilon$	$D \rightarrow \epsilon$	$D \rightarrow \epsilon$

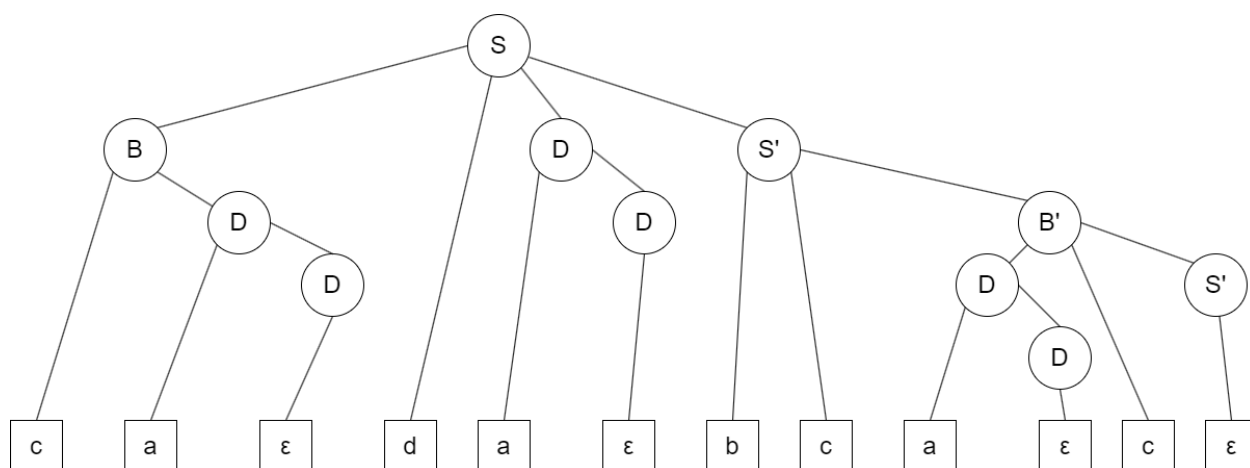


Рис. 1. Пример вывода цепочки

4.3 Синтаксическая диаграмма

На рисунке 2 представлена синтаксическая диаграмма.

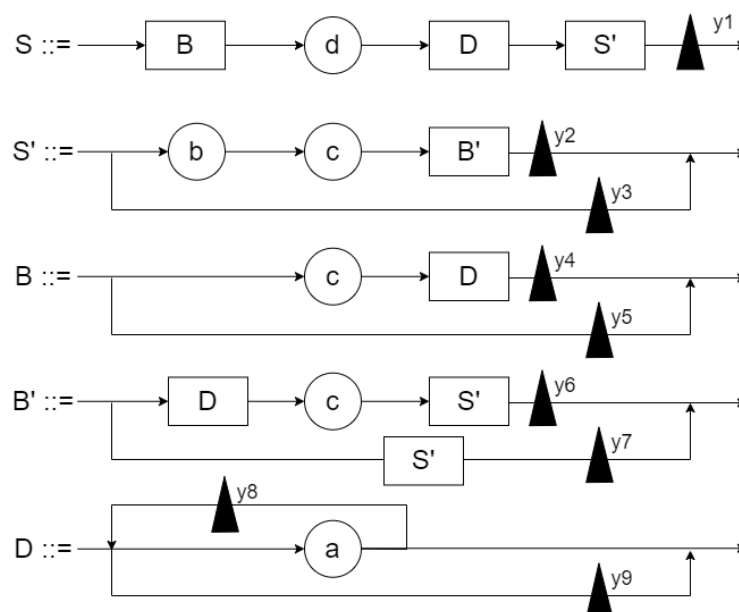


Рис. 2. Синтаксическая диаграмма

4.4 Заданные семантические действия

$S \rightarrow BdDS'$: у1 «Обратиться для покупки абонеента»

$S' \rightarrow bcB'$: у2 «Подписать договор»

$S' \rightarrow \epsilon$: у3 «Оплатить абонемент»

$B \rightarrow cD$: у4 «Выбрать тип абонеента»

$B \rightarrow \epsilon$: у5 «Выбрать тип абонеента по умолчанию (согласно возрасту и мед.показаниям)»

$B' \rightarrow DcS'$: у6 «Добавить дополнительные занятия»

$B' \rightarrow S'$: у7 «Пропустить рекламу дополнительных занятий»

$D \rightarrow aD$: у8 «Добавить дни и время посещения»

$D \rightarrow \epsilon$: у9 «Забронировать место в группе»

5 Особенности реализации

Для реализации детерминированного левого анализатора был создан класс `LL1Grammar`, в котором реализована валидация и генерация цепочек символов. Анализатор использует алгоритм рекурсивного спуска.

5.1 Структуры данных

В классе хранятся следующие структуры данных:

- `start_symbol` — строка - стартовый символ;
- `terminals` — множество терминальных символов;
- `non_terminals` — множество нетерминальных символов;
- `parse_table` — словарь, в котором каждому нетерминалу-ключу соответствует правая часть продукции (таблица выбора);
- `sem_rules` — словарь, в котором каждому правилу-ключу соответствует семантическое действие;
- `gen_rules` — словарь, в котором для каждого нетерминала прописаны возможные правила грамматики.
- `stack` — стек, который начинается с `$` и стартового символа.
- `buf_str` — буфер (список символов входной строки с `$` в обратном порядке).

5.2 Инициализация

Метод `__init__` предназначен для инициализации и хранения множеств терминалов и нетерминалов, стартового символа, а также таблиц семантических действий, выбора и синтаксических правил.

Вход: ссылка на экземпляр класса.

Выход: таблицы семантических действий, выбора и синтаксических правил; стартовый символ, алфавит языка.

Листинг 1. Функция `__init__`

```
1 def __init__(self):
2     self.start_symbol = 'S'
3     self.terminals = {'a', 'b', 'c', 'd', '$'}
4     self.non_terminals = {'S', "S'", 'B', "B'", 'D'}
5
6     self.parse_table = {
7         'S': {'c': ['B', 'd', 'D', "S'"], 'd': ['B', 'd', 'D', "S'"]},
8         "S'": {'b': ['b', 'c', "B"], '$': ['e']},
```

```

9      'B': {'c': ['c', 'D'], 'd': ['e']},
10     'B': {'a': ['D', 'c', "S"], 'c': ['D', 'c', "S"], 'b': ["S"], '$': ["S"]},
11     'D': {'a': ['a', 'D'], 'b': ['e'], 'c': ['e'], 'd': ['e'], '$': ['e']}
12   ]}
13
14   self.sem_rules = {
15     ('S', "BdDS"): "y1 Обратиться<< для покупки абонемена>>",
16     ("S", "bcB"): "y2 Подписать<< договор>>",
17     ("S", "e"): "y3 Оплатить<< абонемент>>",
18     ('B', "cD"): "y4 Выбрать<< тип абонемена>>",
19     ('B', "e"): "y5 Выбрать<< тип абонемена по умолчанию согласно( возрасту и медпоказаниям.)>>",
20     ("B", "DcS"): "y6 Добавить<< дополнительные занятия>>",
21     ("B", "S"): "y7 Пропустить<< рекламу дополнительных занятий>>",
22     ('D', "aD"): "y8 Добавить<< дни и время посещения>>",
23     ('D', "e"): "y9 Забронировать<< место в группе>>"
24   }
25
26   self.gen_rules = {
27     'S': [['B', 'd', 'D', "S"]],
28     "S": [['b', 'c', "B"], []],
29     'B': [['c', 'D'], []],
30     "B": [['D', 'c', "S"], ["S"]],
31     'D': [['a', 'D'], []]
32   }
33

```

5.3 Генератор цепочек

Метод `generate` предназначен для генерации цепочки в соответствии с правилами грамматики. Он использует рекурсивный алгоритм с ограничением сверху параметром `max_depth`. Вложенная функция `add` принимает текущий символ и оставшуюся глубину. Если `max_depth` равен нулю, то возвращается пустая строка. Если символ - терминал, то он возвращается без изменений. Если символ - нетерминал, то для него случайно выбирается одно из правил правой части, которое тоже рекурсивно раскрывается с уменьшением `max_depth` на единицу. В случае, когда в качестве правила выбирается пустая строка, она просто возвращается.

Вход: ссылка на экземпляр класса, максимальная глубина рекурсии.

Выход: сгенерированная цепочка символов.

Листинг 2. `generate`

```

1     def generate(self, max_depth=8):
2     def add(symbol, depth):
3     if depth <= 0:
4     return ''
5     if symbol in self.terminals:
6     return symbol
7     rhs = random.choice(self.gen_rules[symbol])
8     if not rhs:
9     return ''
10    return ''.join(add(s, depth - 1) for s in rhs)

```

```

11
12     return add(self.start_symbol, max_depth)
13

```

5.4 Анализатор цепочек

Метод `validate` предназначен для проверки принадлежности цепочки правилам грамматики. На каждой итерации цикла используется верхушка стека и текущий символ буфера. Пока верхний символ стека не равен `$`, сравниваем его с текущим символом. Если они совпадают, переходим к следующему символу и удаляем из стека. Если верхушка является терминалом, то по таблице выбора ищем правило для пары (вершина, текущий символ) и заменяем вершину на правую часть правила, применяя при этом семантическое действие. Если текущая вершина равна `$`, то выходим из цикла с успехом, выводя все накопленные семантические действия. При ошибке также выходим из цикла, но с булевым флагом `False` и описанием ошибки.

Вход: ссылка на экземпляр класса, цепочка для анализа.

Выход: булевый флаг, указывающий на корректность цепочки, сообщение об успехе или ошибке.

Листинг 3. `validate`

```

1     def validate(self, text):
2 input_buf = list(text.strip())
3 stack = ['$ ', self.start_symbol]
4 pos = 0
5 triggered_actions = []
6
7 print(f"{'Stack':<18} | {'Buffer':<18} | {'Rule | Action'}")
8
9 while stack:
10 stack_str = ''.join(stack[::-1])
11 remaining = ''.join(input_buf[pos:])
12 buf_str = '$ ' + remaining[::-1]
13 top = stack.pop()
14 cur = input_buf[pos] if pos < len(input_buf) else '$ '
15
16 if top == cur:
17 if top == '$ ':
18 print(f"{'stack_str':<18} | {'buf_str':<18} | Valid")
19 print("\nSemantic rules:")
20 for act in triggered_actions:
21 print(f"    {act}")
22 return True, "Valid string."
23
24 pos += 1
25 print(f"{'stack_str':<18} | {'buf_str':<18} | Match '{cur}'")
26
27 elif top in self.terminals:
28 print(f"{'stack_str':<18} | {'buf_str':<18} | Error: expected '{top}', input '{cur}'")
29 return False, f"Error {pos}: expected '{top}'"

```

```

30
31 else: # нетерминал
32 rule = self.parse_table.get(top, {}).get(cur)
33 if rule is None:
34 print(f"{stack_str:<18} | {buf_str:<18} | Error: there is no such rule for
    '{top}' with '{cur}'")
35 return False, f"There is no such rule for '{top}' with '{cur}'"
36 rule_str = ''.join(rule)
37 if rule != ['e']:
38 stack.extend(rule[::-1])
39 action = self.sem_rules.get((top, rule_str), "")
40 if action:
41 triggered_actions.append(action)
42 print(f"{stack_str:<18} | {buf_str:<18} | {top} -> {rule_str} | {action}")
43 else:
44 action = self.sem_rules.get((top, 'e'), "")
45 if action:
46 triggered_actions.append(action)
47 print(f"{stack_str:<18} | {buf_str:<18} | {top} -> e | {action}")
48 return False, "Unexpected end"
49

```

6 Результаты работы программы

Результаты выполнения представлены на рисунках 3- 7.

Stack	Buffer	Rule	Action
S\$	\$daabcc	S -> BdDS' y1 <<Обратиться для покупки абонеента>>	
BdDS'\$	\$daabcc	B -> ε y5 <<Выбрать тип абонеента по умолчанию (согласно возрасту и мед.показаниям)>>	
dDS'\$	\$daabcc	Match 'd'	
DS'\$	\$aabcc	D -> aD y8 <<Добавить дни и время посещения>>	
aDS'\$	\$aabcc	Match 'a'	
DS'\$	\$abcc	D -> aD y8 <<Добавить дни и время посещения>>	
aDS'\$	\$abcc	Match 'a'	
DS'\$	\$bcc	D -> ε y9 <<Забронировать место в группе>>	
S'\$	\$bcc	S' -> bcB' y2 <<Подписать договор>>	
bcB'\$	\$bcc	Match 'b'	
cB'\$	\$cc	Match 'c'	
B'\$	\$c	B' -> DcS' y6 <<Добавить дополнительные занятия>>	
DcS'\$	\$c	D -> ε y9 <<Забронировать место в группе>>	
cS'\$	\$c	Match 'c'	
S'\$	\$	S' -> ε y3 <<Оплатить абонемент>>	
\$	\$	Valid	

Semantic rules:

- y1 <<Обратиться для покупки абонеента>>
- y5 <<Выбрать тип абонеента по умолчанию (согласно возрасту и мед.показаниям)>>
- y8 <<Добавить дни и время посещения>>
- y8 <<Добавить дни и время посещения>>
- y9 <<Забронировать место в группе>>
- y2 <<Подписать договор>>
- y6 <<Добавить дополнительные занятия>>
- y9 <<Забронировать место в группе>>
- y3 <<Оплатить абонемент>>

Random sentence: daabcc -> [correct]

Рис. 3. Генерация случайной строки

Input sentence: dbcc			
Stack	Buffer	Rule	Action
S\$	\$dbcc	S -> BdDS' y1 <<Обратиться для покупки абонеента>>	
BdDS'\$	\$dbcc	B -> ε y5 <<Выбрать тип абонеента по умолчанию (согласно возрасту и мед.показаниям)>>	
dDS'\$	\$dbcc	Match 'd'	
DS'\$	\$bcc	D -> ε y9 <<Забронировать место в группе>>	
S'\$	\$bcc	S' -> bcB' y2 <<Подписать договор>>	
bcB'\$	\$bcc	Match 'b'	
cB'\$	\$cc	Match 'c'	
B'\$	\$c	B' -> DcS' y6 <<Добавить дополнительные занятия>>	
DcS'\$	\$c	D -> ε y9 <<Забронировать место в группе>>	
cS'\$	\$c	Match 'c'	
S'\$	\$	S' -> ε y3 <<Оплатить абонемент>>	
\$	\$	Valid	

Semantic rules:

- y1 <<Обратиться для покупки абонеента>>
- y5 <<Выбрать тип абонеента по умолчанию (согласно возрасту и мед.показаниям)>>
- y9 <<Забронировать место в группе>>
- y2 <<Подписать договор>>
- y6 <<Добавить дополнительные занятия>>
- y9 <<Забронировать место в группе>>
- y3 <<Оплатить абонемент>>

Result: correct

Рис. 4. Корректно введенная строка

```

Input sentence: dbaa
Stack          | Buffer          | Rule | Action
S$             | $dbaa          | S -> BdDS' | y1 <<Обратиться для покупки абонеента>>
BdDS'$         | $dbaa          | B -> ε | y5 <<Выбрать тип абонеента по умолчанию (согласно возрасту и мед.показаниям)>>
dDS'$          | $dbaa          | Match 'd'
DS'$           | $baa           | D -> ε | y9 <<Забронировать место в группе>>
S'$            | $baa           | S' -> bcB' | y2 <<Подписать договор>>
bcB'$          | $baa           | Match 'b'
cB'$           | $aa            | Error: expected 'c', input 'a'
Result: error

```

Рис. 5. Некорректно введенная строка

```

Input sentence: cdabocaaac
Stack          | Buffer          | Rule | Action
S$             | $cdabocaaac    | S -> BdDS' | y1 <<Обратиться для покупки абонеента>>
BdDS'$         | $cdabocaaac    | B -> cD | y4 <<Выбрать тип абонеента>>
cDdDS'$        | $cdabocaaac    | Match 'c'
DdDS'$         | $dabocaaac     | D -> ε | y9 <<Забронировать место в группе>>
dDS'$          | $dabocaaac     | Match 'd'
DS'$           | $abocaaac      | D -> aD | y8 <<Добавить дни и время посещения>>
aDS'$          | $abocaaac      | Match 'a'
DS'$           | $bocaaac       | D -> ε | y9 <<Забронировать место в группе>>
S'$            | $bocaaac       | S' -> bcB' | y2 <<Подписать договор>>
bcB'$          | $bocaaac       | Match 'b'
cB'$           | $ocaaac        | Error: 'o' not in input alphabet: {'d', 'a', 'b', '$', 'c'}
Result: error

```

Рис. 6. Строка с символами не из словаря

```

Input sentence:
Stack          | Buffer          | Rule | Action
S$             | $              | Error: there is no such rule for 'S' with '$'
Result: error

```

Рис. 7. Пустая строка

Заключение

В ходе выполнения лабораторной работы для заданной грамматики были построены множества FIRST и FOLLOW для каждого нетерминала грамматики, а также таблица выбора и матрица отношений предшествования. Был проведен анализ на принадлежность грамматики к конкретному типу. В итоге грамматика была упрощена до вида LL(1), анализатор грамматики использует метод рекурсивного спуска. Каждой продукции были назначены семантические действия, принадлежащие предметной области «Оформление абонемента в бассейн». На основе таблицы выбора был успешно реализован детерминированный левый анализатор и генератор цепочек. Реализованная программа была проверена на наборе примеров, включающем как корректные, так и некорректные входные данные.

Достоинства программы:

- алгоритм работает за линейное время;
- генератор использует ограничение глубины рекурсии для защиты от заикливания;
- вывод логов распознавания достаточен для отслеживания работы пошагово.

Недостатки программы:

- правила грамматики заданы в коде, их нельзя динамически изменять;
- поддерживаются только конструкции, применимые к LL(1) грамматике (не работает с леворекурсивными правилами).

Масштабирование программы:

- замена текстовых семантических действий на функции, которые выполняют действия;
- автоматическое вычисление множеств FIRST и FOLLOW на основе правил.

Список использованной литературы

- [1] Секция «Телематика». — Текст : электронный // tema.spbstu.ru : [сайт]. URL: <https://tema.spbstu.ru/mathem/> (дата обращения: 29.04.2026).
- [2] Сети, Р.; Ахо, А. Компиляторы: принципы, технологии и инструментарий - М.: Издательство «Наука», 2008. - 1178 с. (дата обращения: 29.04.2026).
- [3] Ю.Г. Карпов Основы построения трансляторов. - СПб: БХВ-Петербург, 2005. - 272 с. (дата обращения - 12.05.2026).